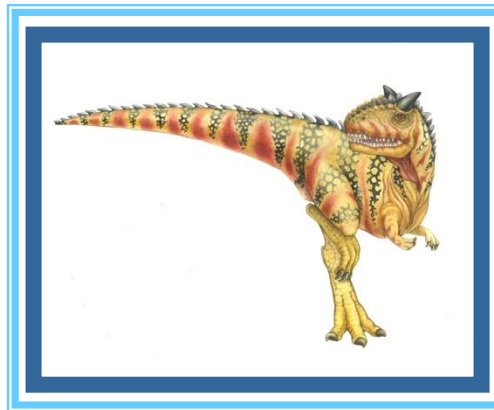


Servicios del sistema operativo, diseño e implementación





¿Que servicios puede tener un sistema operativo?





Servicios del sistema operativo

- Los sistemas operativos proporcionan un entorno para la ejecución de programas y servicios a programas y usuarios.
- Un conjunto de servicios del sistema operativo proporciona funciones que son útiles para el usuario:
 - **Interfaz de usuario** : Casi todos los sistemas operativos tienen una interfaz de usuario (**UI**).
 - ▶ Varía entre **la línea de comandos (CLI)** , **Gráficos Usuario Interfaz (GUI)** , **pantalla táctil , batch**
 - **Ejecución del programa** : El sistema debe poder cargar un programa en la memoria y ejecutar ese programa, finalizar la ejecución, ya sea de manera normal o anormal (indicando un error).
 - **Operaciones de I/O** : un programa en ejecución puede requerir I/O, que puede involucrar un archivo o un dispositivo de I/O.
 - **Manipulación del sistema de archivos** : El sistema de archivos es de especial interés. Los programas necesitan leer y escribir archivos y directorios, crearlos y eliminarlos, buscarlos, listar información de archivos y gestionar permisos.





Servicios del sistema operativo (cont.)

- Un conjunto de servicios del sistema operativo proporciona funciones que son útiles para el usuario (Cont.):
 - **Comunicaciones** : Los procesos pueden intercambiar información, en la misma computadora o entre computadoras a través de una red.
 - ▶ Las comunicaciones pueden ser a través de memoria compartida o mediante paso de mensajes (paquetes movidos por el sistema operativo).
 - **Detección de errores** : el sistema operativo debe estar constantemente al tanto de posibles errores.
 - ▶ Puede ocurrir en el hardware de la CPU y la memoria, en los dispositivos de I/O, en el programa del usuario.
 - ▶ Para cada tipo de error, el sistema operativo debe tomar las medidas adecuadas para garantizar un procesamiento correcto y consistente.
 - ▶ Las funciones de depuración pueden mejorar enormemente las capacidades del usuario y del programador para utilizar el sistema de manera eficiente.





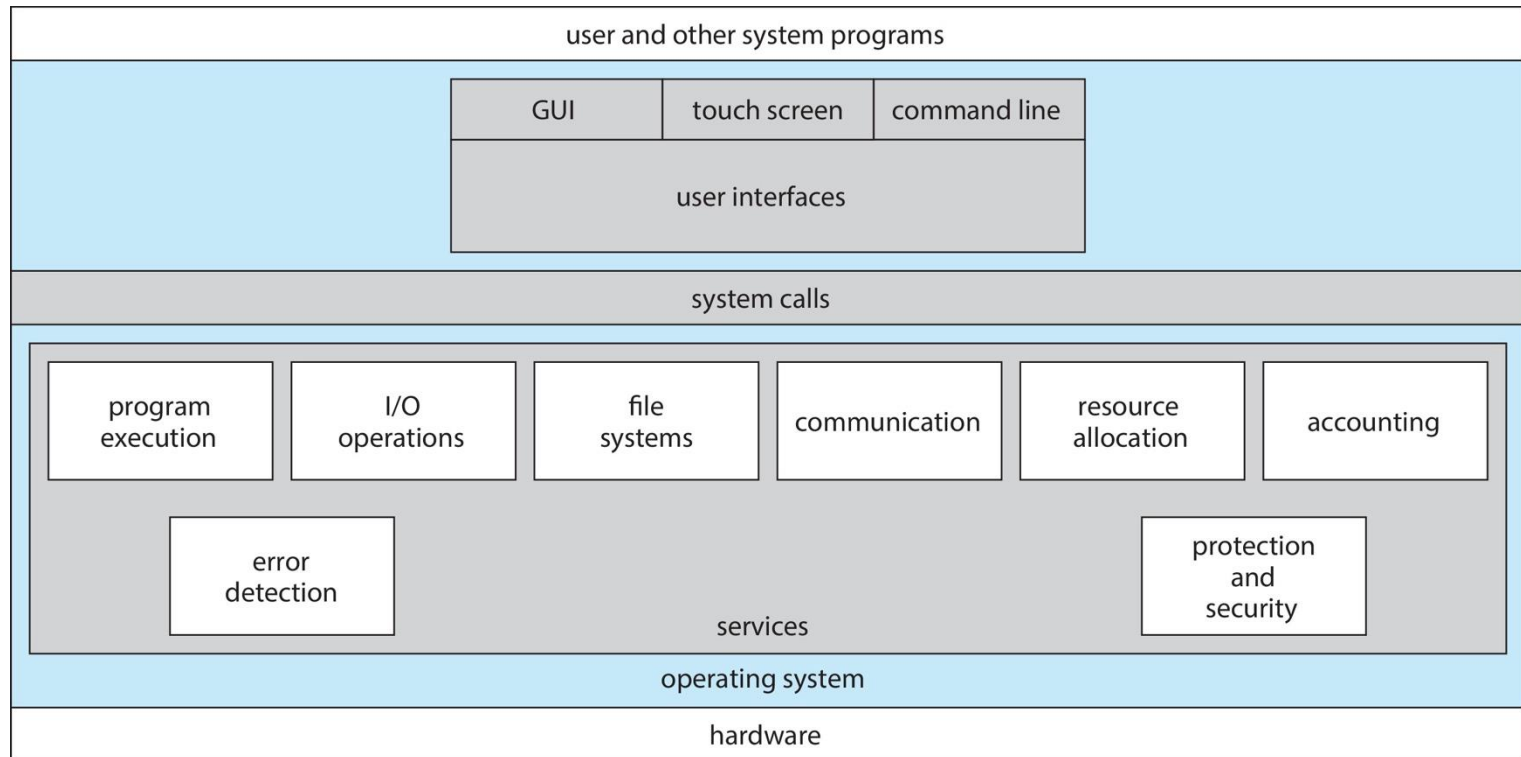
Servicios del sistema operativo (cont.)

- Existe otro conjunto de funciones del sistema operativo para garantizar el funcionamiento eficiente del propio sistema mediante el uso compartido de recursos.
 - **Asignación de recursos:** cuando varios usuarios o varios trabajos se ejecutan simultáneamente, se deben asignar recursos a cada uno de ellos.
 - ▶ Muchos tipos de recursos: ciclos de CPU, memoria principal, almacenamiento de archivos, dispositivos de I/O.
 - **Registro:** para realizar un seguimiento de qué usuarios utilizan cuánto y qué tipo de recursos informáticos
 - **Protección y seguridad** - Los propietarios de información almacenada en un sistema informático multiusuario o en red pueden querer controlar el uso de esa información; los procesos concurrentes no deben interferir entre sí.
 - ▶ **La protección** implica garantizar que todo acceso a los recursos del sistema esté controlado.
 - ▶ **La seguridad** del sistema frente a elementos externos requiere la autenticación del usuario y se extiende a la defensa de los dispositivos de I/O externos frente a intentos de acceso no válidos.





Una visión de los servicios del sistema operativo





Intérprete de línea de comandos

- La CLI permite la entrada directa de comandos
- A veces implementado en el kernel, a veces mediante programas de sistemas.
- A veces se implementan múltiples sabores: **shells**
- Principalmente obtiene un comando del usuario y lo ejecuta.
- A veces comandos integrados, a veces solo nombres de programas
 - Si es esto último, agregar nuevas funciones no requiere modificar el shell.

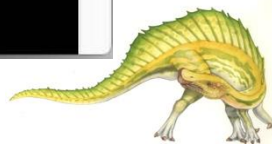




Intérprete de comandos de Bourne Shell

```
1. root@r6181-d5-us01:~ (ssh)
× root@r6181-d5-u...  %1 × ssh  %2 × root@r6181-d5-us01...  %3

Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root    50G       19G   28G  41% /
tmpfs                      127G      520K   127G   1% /dev/shm
/dev/sda1                   477M       71M   381M  16% /boot
/dev/dssd0000               1.0T     480G   545G  47% /dssd_xfs
tcp://192.168.150.1:3334/orangeifs
                          12T     5.7T   6.4T  47% /mnt/orangeifs
/dev/gpfs-test              23T     1.1T    22T   5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2   6.6 42665344 17520636 ?        S<Ll    Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root      69849   6.6   0.0      0      0 ?        S       Jul12 181:54 [vpthread-1-1]
root      69850   6.4   0.0      0      0 ?        S       Jul12 177:42 [vpthread-1-2]
root       3829   3.0   0.0      0      0 ?        S       Jun27 730:04 [rp_thread 7:0]
root       3826   3.0   0.0      0      0 ?        S       Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```





Interfaz del sistema operativo del usuario - GUI

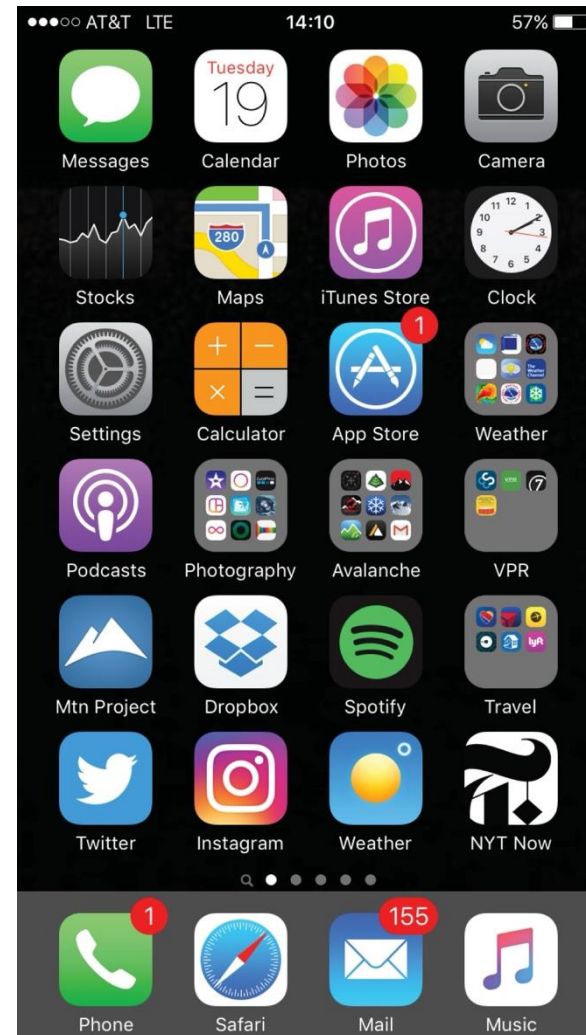
- Interfaz de metáfora **de escritorio** fácil de usar
 - Generalmente ratón, teclado y monitor.
 - **Los iconos** representan archivos, programas, acciones, etc.
 - Varios botones del mouse sobre objetos en la interfaz provocan varias acciones (proporcionar información, opciones, ejecutar funciones, abrir un directorio (conocido como **carpeta**)
 - Inventado en Xerox PARC
- Muchos sistemas ahora incluyen interfaces CLI y GUI
 - Microsoft Windows es una GUI con shell de “ comandos ” CLI
 - Apple Mac OS X es una interfaz GUI “ Aqua ” con kernel UNIX debajo y shells disponibles
 - Unix y Linux tienen CLI con interfaces GUI opcionales (CDE, KDE, GNOME)





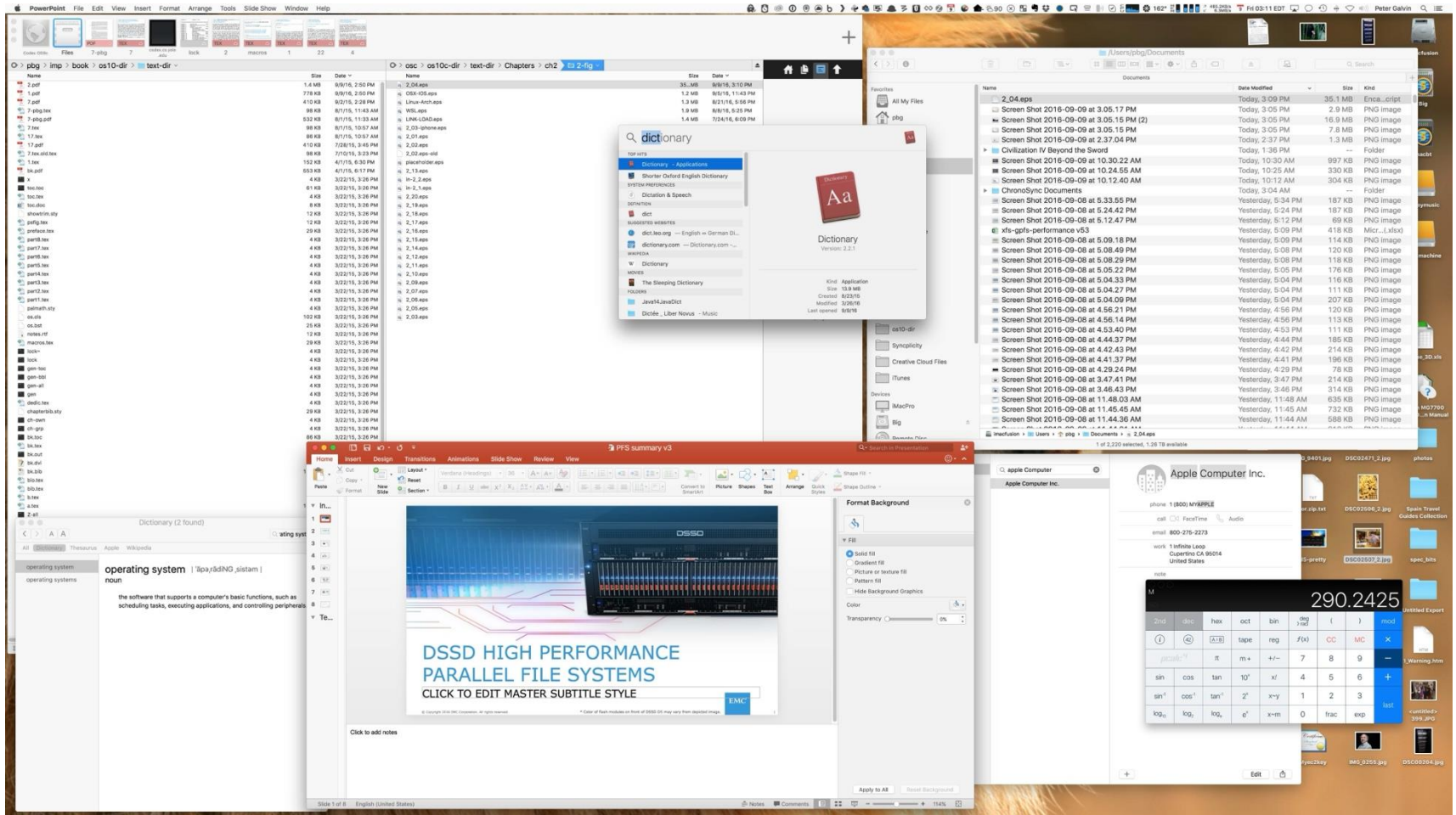
Interfaces de pantalla táctil

- Los dispositivos de pantalla táctil requieren nuevas interfaces
 - Ratón no posible o no deseado
 - Acciones y selección basadas en gestos
 - Teclado virtual para entrada de texto
- Comandos de voz





La interfaz gráfica de usuario de Mac OS X





¿Llamadas al sistema?

¿Cómo se programan?

**¿Qué tipos de llamadas al sistema
podríamos tener?**





Llamadas al sistema

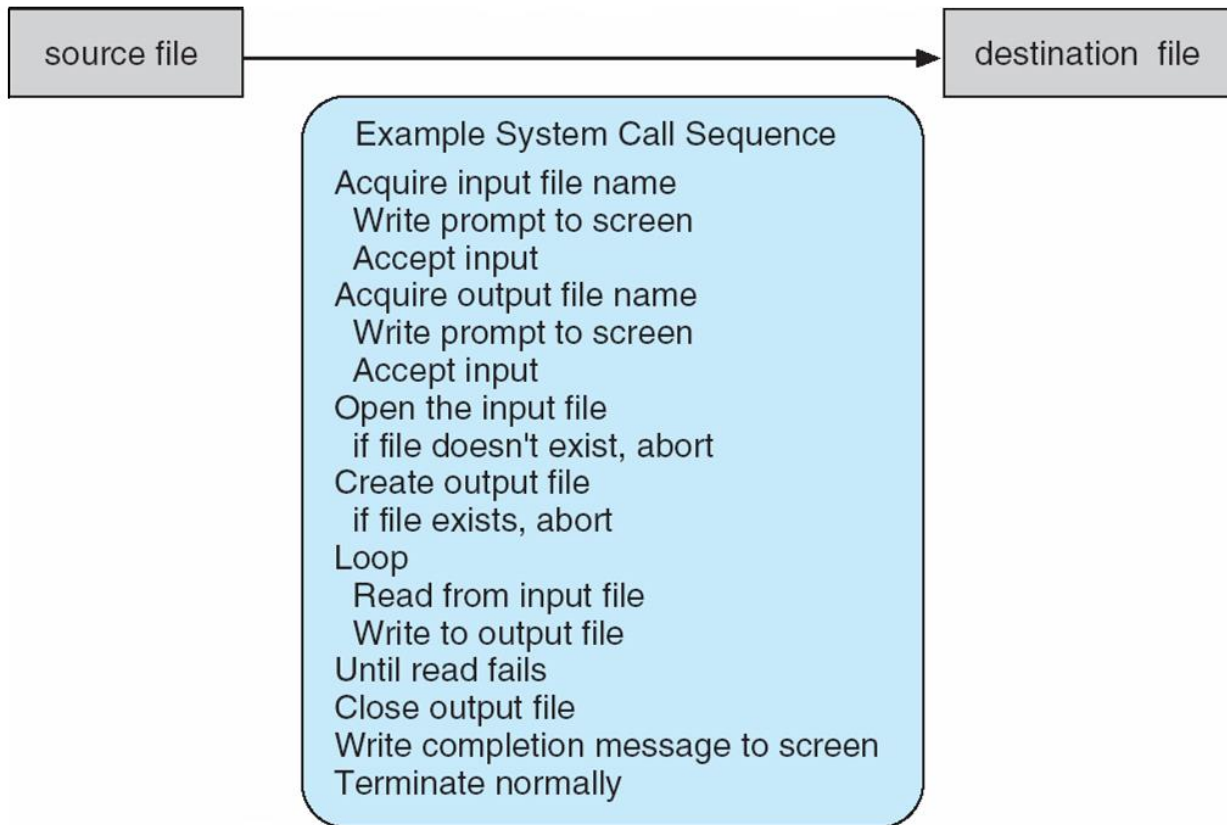
- Interfaz de programación para los servicios proporcionados por el sistema operativo
- Generalmente escrito en un lenguaje de alto nivel (C o C++)
- **aplicación** de alto nivel. **Programación Interfaz** (**API**) en lugar del uso directo de llamadas al sistema
- Las tres API más comunes son la API Win32 para Windows, la API POSIX para sistemas basados en POSIX (incluidas prácticamente todas las versiones de UNIX, Linux y Mac OS X) y la API Java para la máquina virtual Java (JVM).





Ejemplo de llamadas al sistema

- Secuencia de llamada del sistema para copiar el contenido de un archivo a otro archivo





Ejemplo de API estándar

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the man page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

#include <unistd.h>		
ssize_t	read	(int fd, void *buf, size_t count)
return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.





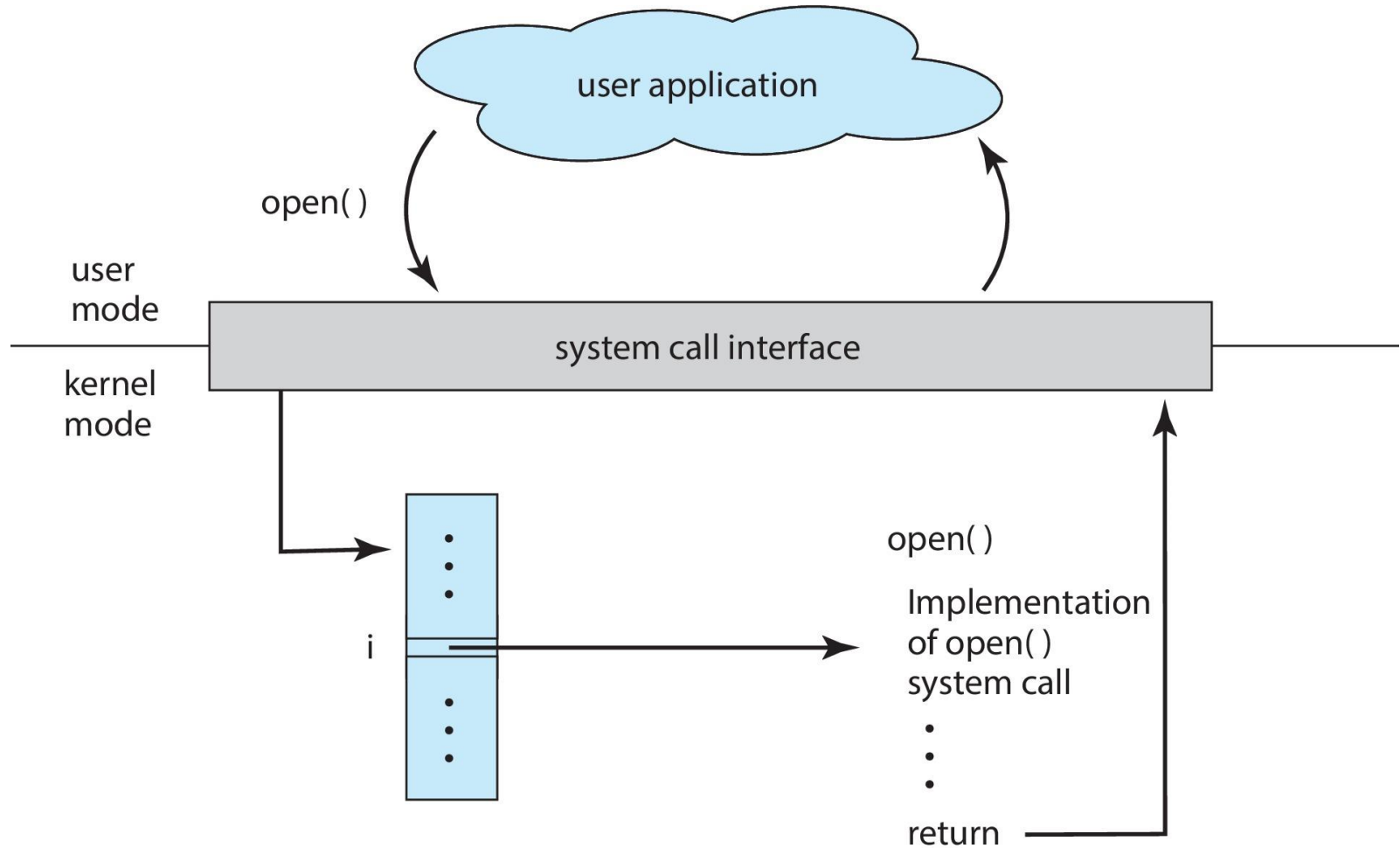
Implementación de llamadas al sistema

- Normalmente, se asocia un número con cada llamada del sistema.
 - **Interfaz de llamadas a sistema** Mantiene una tabla indexada según estos números
- La interfaz de llamada del sistema invoca la llamada del sistema prevista en el kernel del sistema operativo y devuelve el estado de la llamada del sistema y cualquier valor de retorno.
- El llamante no necesita saber nada sobre cómo se implementa la llamada al sistema.
 - Solo necesita obedecer la API y comprender qué hará el sistema operativo como resultado de la llamada.
 - La mayoría de los detalles de la interfaz del sistema operativo están ocultos para el programador por la API
 - ▶ Administrado por la biblioteca de soporte en tiempo de ejecución (conjunto de funciones integradas en bibliotecas incluidas con el compilador)





Relación API – Llamada del sistema – Sistema Operativo





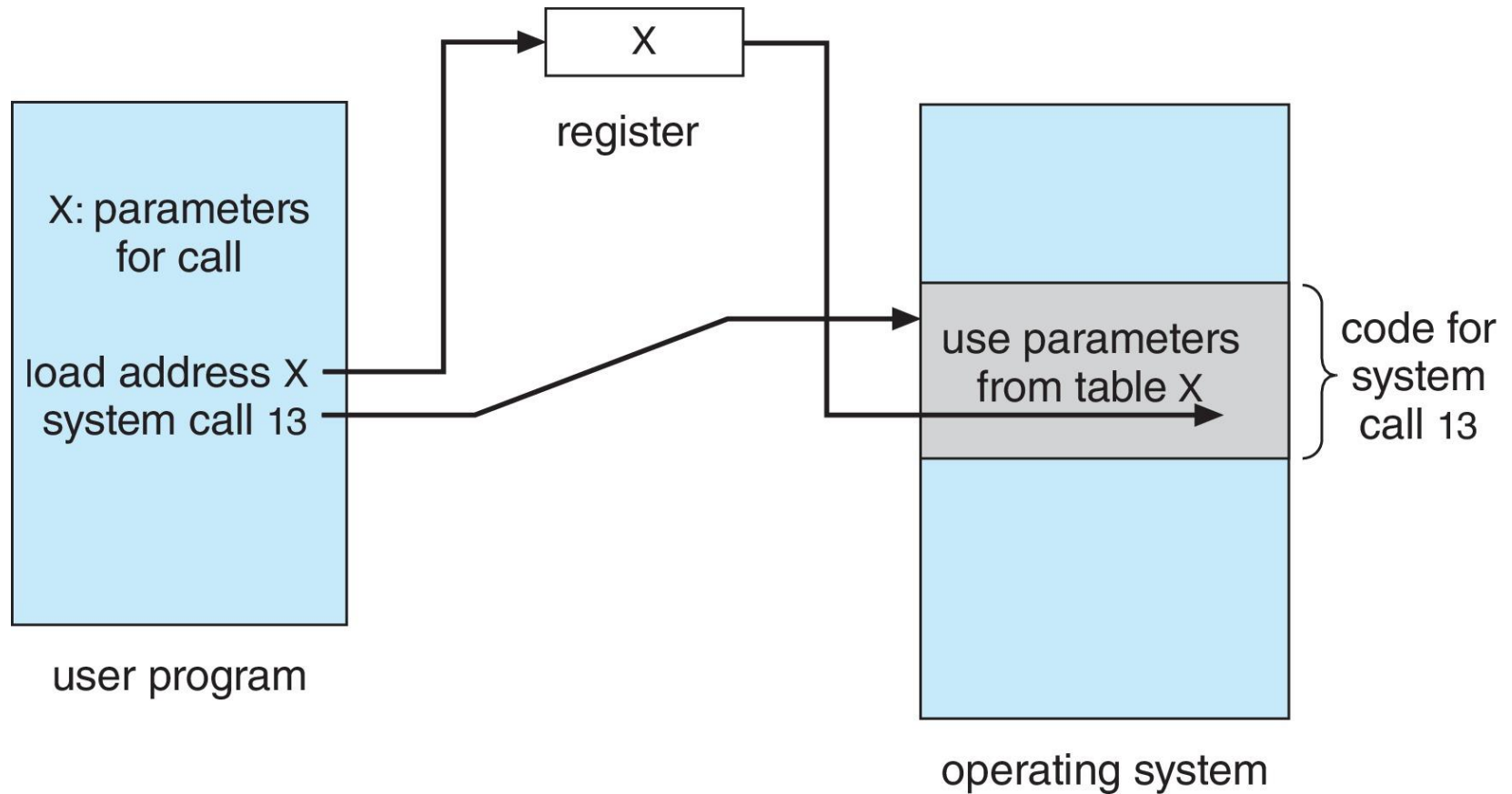
Paso de parámetros de llamada del sistema

- A menudo, se requiere más información que simplemente la identidad de la llamada al sistema deseada.
 - El tipo exacto y la cantidad de información varían según el sistema operativo y la llamada.
- Tres métodos generales utilizados para pasar parámetros al sistema operativo
 - Lo más simple: pasar los parámetros en registros
 - ▶ En algunos casos, puede haber más parámetros que registros.
 - Parámetros almacenados en un bloque o tabla en la memoria y dirección del bloque pasada como parámetro en un registro
 - ▶ Este enfoque es adoptado por Linux y Solaris
 - Parámetros colocados o **insertados** en la **pila** por el programa y **expulsados** fuera de la pila por el sistema operativo
 - Los métodos de bloqueo y pila no limitan la cantidad ni la longitud de los parámetros que se pasan





Paso de parámetros a través de la tabla





Tipos de llamadas al sistema

- Control de procesos
 - crear proceso, terminar proceso
 - fin, abortar
 - cargar, ejecutar
 - obtener atributos del proceso, establecer atributos del proceso
 - esperar el tiempo
 - evento de espera, evento de señal
 - asignar y liberar memoria
 - Volcar memoria en caso de error
 - **Debugger** para determinar **errores** , ejecución **de un paso**
 - **Locks** para gestionar el acceso a datos compartidos entre procesos





Tipos de llamadas al sistema (cont.)

- Gestión de archivos
 - crear archivo, eliminar archivo
 - abrir, cerrar archivo
 - leer, escribir, reposicionar
 - obtener y establecer atributos de archivo
- Gestión de dispositivos
 - Solicitud de dispositivo, liberación de dispositivo
 - leer, escribir, reposicionar
 - obtener atributos del dispositivo, establecer atributos del dispositivo
 - conectar o desconectar dispositivos de forma lógica

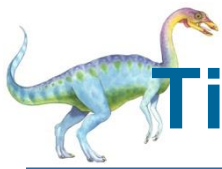




Tipos de llamadas al sistema (cont.)

- Mantenimiento de la información
 - obtener hora o fecha, establecer hora o fecha
 - obtener datos del sistema, establecer datos del sistema
 - Obtener y establecer atributos de proceso, archivo o dispositivo
- Comunicaciones
 - crear, eliminar conexión de comunicación
 - enviar, recibir mensajes si se usa el **modelo de paso de mensajes** al **nombre de host** o **nombre del proceso**
 - ▶ De **cliente** a **servidor**
 - **Modelo de memoria compartida**, crear y obtener acceso a regiones de memoria
 - información del estado de la transferencia
 - conectar y desconectar dispositivos remotos





Tipos de llamadas al sistema (cont.)

- Protección
 - Controlar el acceso a los recursos
 - Obtener y establecer permisos
 - Permitir y denegar el acceso de usuarios





Ejemplos de llamadas al sistema de Windows y Unix

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()



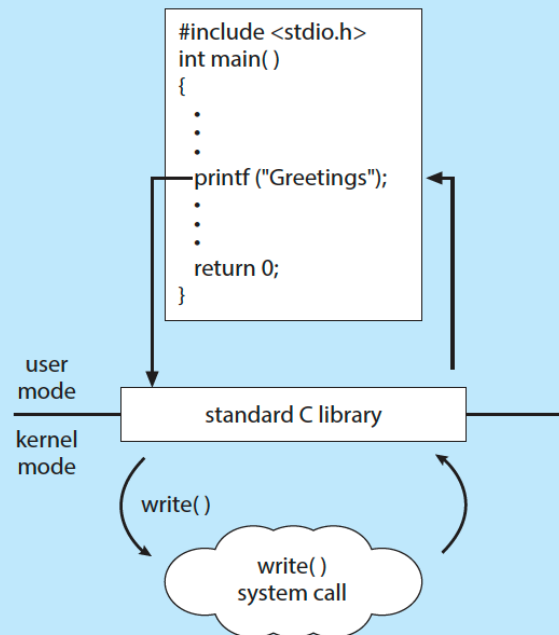


Ejemplo de biblioteca C estándar

- Programa en C que invoca la llamada a la biblioteca `printf()`, que llama a la llamada al sistema `write()`

THE STANDARD C LIBRARY

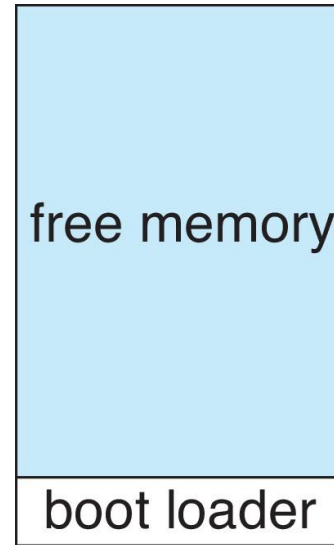
The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



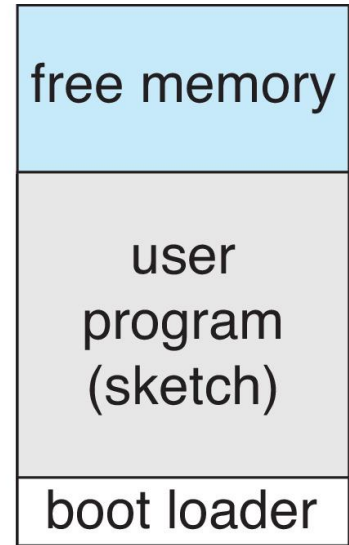


Ejemplo: Arduino

- Una sola tarea
- Sin sistema operativo
- Programas (sketch) cargados vía USB en la memoria flash
- Espacio de memoria único
- El cargador de arranque carga el programa
- Salida del programa -> shell recargado



(a)



(b)

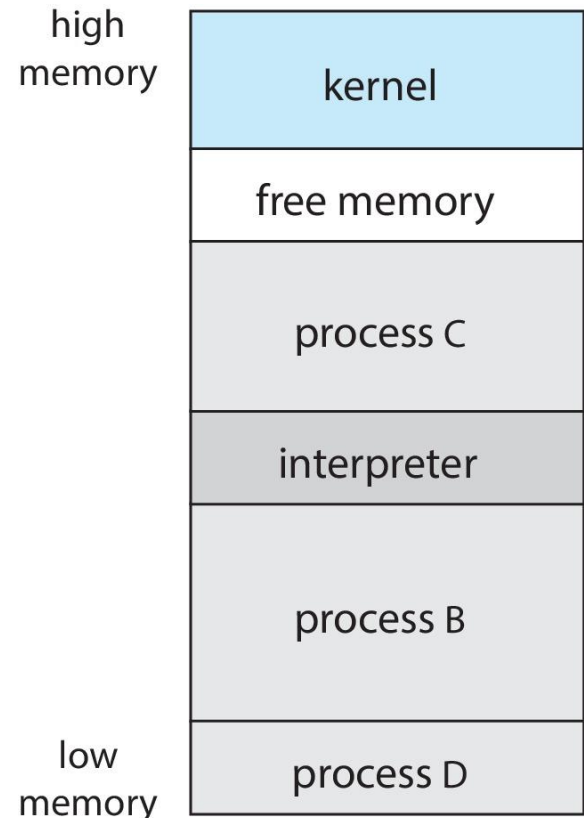
Al iniciar el sistema se ejecuta un programa





Ejemplo: FreeBSD

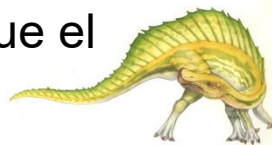
- Variante de Unix
- Multitarea
- Inicio de sesión de usuario -> invocar la elección de shell del usuario
- Shell ejecuta la llamada al sistema fork() para crear el proceso
 - Ejecuta exec() para cargar el programa en el proceso
 - Shell espera a que el proceso finalice o continúa con los comandos del usuario
- El proceso sale con:
 - código = 0 – sin error
 - código > 0 – código de error





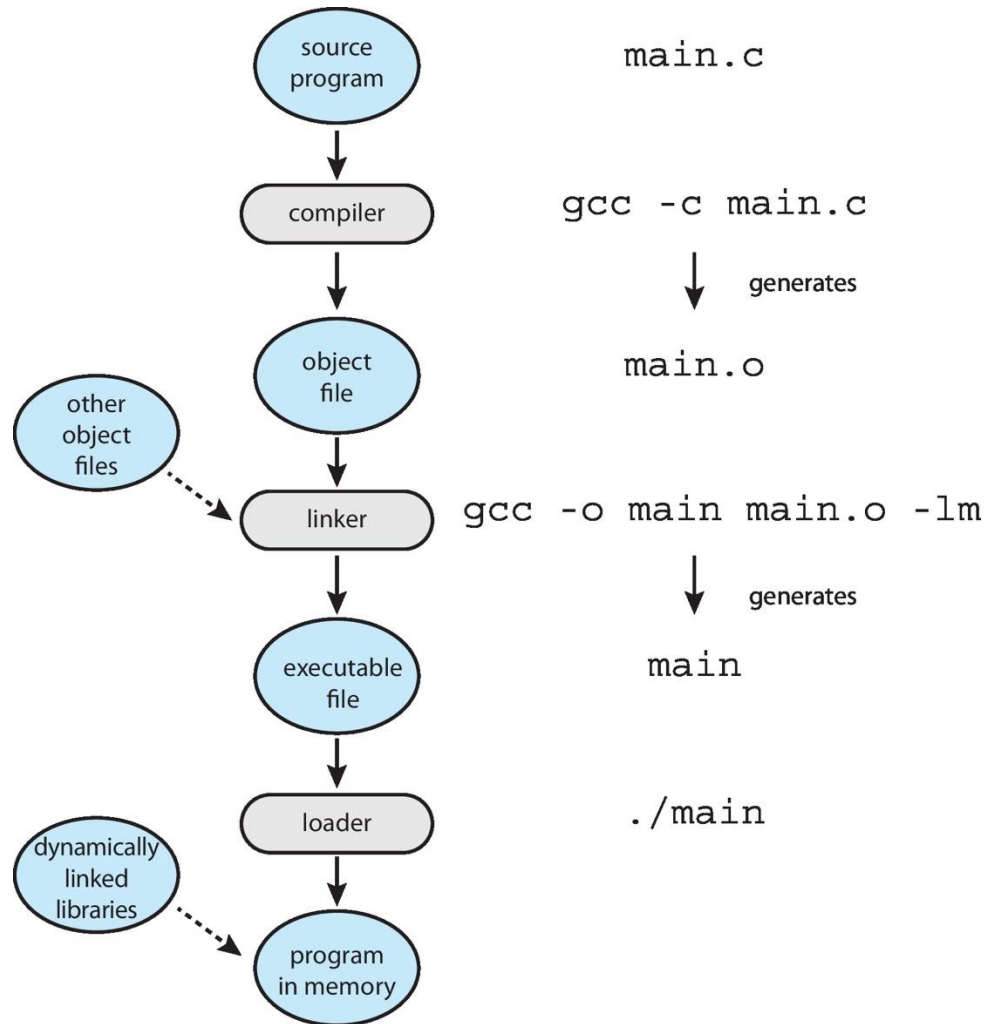
Linkers y Loaders

- Código fuente compilado en archivos de objeto diseñados para cargarse en cualquier ubicación de memoria física – **archivo de objeto reubicable**
- **Linker** Combina todo esto en un único archivo binario **ejecutable**.
 - También incluye bibliotecas
- El programa reside en el almacenamiento secundario como ejecutable binario
- Debe ser llevado a la memoria por **el Loader** para ser ejecutado
 - **La reubicación** asigna direcciones finales a las partes del programa y ajusta el código y los datos del programa para que coincidan con esas direcciones.
- Los sistemas modernos de propósito general no vinculan bibliotecas a ejecutables
 - Más bien, **bibliotecas dinamicamente vinculadas** (en Windows, **las DLL**) se cargan según sea necesario y se comparten entre todos los que usan la misma versión de esa misma biblioteca (se carga una vez)
- Los objetos y archivos ejecutables tienen formatos estándar, por lo que el sistema operativo sabe cómo cargarlos e iniciarlos.





El papel del enlazador y del cargador





¿Por qué las aplicaciones son específicas del sistema operativo?





Por qué las aplicaciones son específicas del sistema operativo

- Las aplicaciones compiladas en un sistema generalmente no son ejecutables en otros sistemas operativos
- Cada sistema operativo proporciona sus propias llamadas de sistema únicas
 - Formatos de archivos propios, etc.
- Las aplicaciones pueden ser multisistema operativo
 - Escrito en lenguaje interpretado como Python, Ruby y con intérprete disponible en múltiples sistemas operativos.
 - Aplicación escrita en un lenguaje que incluye una máquina virtual que contiene la aplicación en ejecución (como Java)
 - Utilice un lenguaje estándar (como C), compile por separado en cada sistema operativo para ejecutarlo en cada uno
- **Interfaz binaria de aplicaciones (ABI)** es el equivalente arquitectónico de la API y define cómo los diferentes componentes del código binario pueden interactuar para un sistema operativo determinado en una arquitectura, CPU, etc. determinados.





¿Cómo se diseña y se implementa un OS?





Diseño e implementación

- El diseño y la implementación del sistema operativo no es “*resolvable*”, pero algunos enfoques han demostrado ser exitosos.
- La estructura interna de diferentes sistemas operativos puede variar ampliamente
- Se inicia el diseño definiendo objetivos y especificaciones
- Afectado por la elección del hardware y el tipo de sistema.
- Objetivos de **Usuario** y objetivos de **Sistema**
 - Objetivos del usuario: el sistema operativo debe ser cómodo de usar, fácil de aprender, confiable, seguro y rápido.
 - Objetivos del sistema: el sistema operativo debe ser fácil de diseñar, implementar y mantener, además de flexible, confiable, libre de errores y eficiente.
- Especificar y diseñar un sistema operativo es una tarea altamente creativa de **Ingeniería de Software**





Políticas y mecanismos

- **Política** : ¿Qué es necesario hacer?
 - Ejemplo: Interrumpir cada 100 segundos
- **Mecanismo** : ¿Cómo hacer algo?
 - Ejemplo: temporizador (Timer)
- Principio importante: separar la política del mecanismo
- La separación entre política y mecanismo es un principio muy importante, ya que permite máxima flexibilidad si las decisiones políticas se deben modificar más adelante.
 - Ejemplo: cambiar 100 a 200





Implementación

- Es muy variable
 - Los primeros sistemas operativos se desarrollaron en lenguaje ensamblador
 - Luego están los lenguajes de programación de sistemas como Algol, PL/1
 - Ahora C, C++
- En realidad, normalmente es una mezcla de lenguajes.
 - Los niveles más bajos en ensamblador
 - El cuerpo principal (main) en C
 - Programas de sistemas en C, C++, lenguajes de script como PERL, Python, scripts de shell
- Un lenguaje de alto nivel más fácil de “portear” a otro hardware
 - Pero más lento
- **La emulación** puede permitir que un sistema operativo se ejecute en hardware no nativo





Estructura del sistema operativo

- El sistema operativo de propósito general es un programa muy grande
- Diversas formas de estructurarlos
 - Estructura simple – MS-DOS
 - Más complejo – UNIX
 - Capas – una abstracción
 - Microkernel – Mach





Estructura monolítica – UNIX original

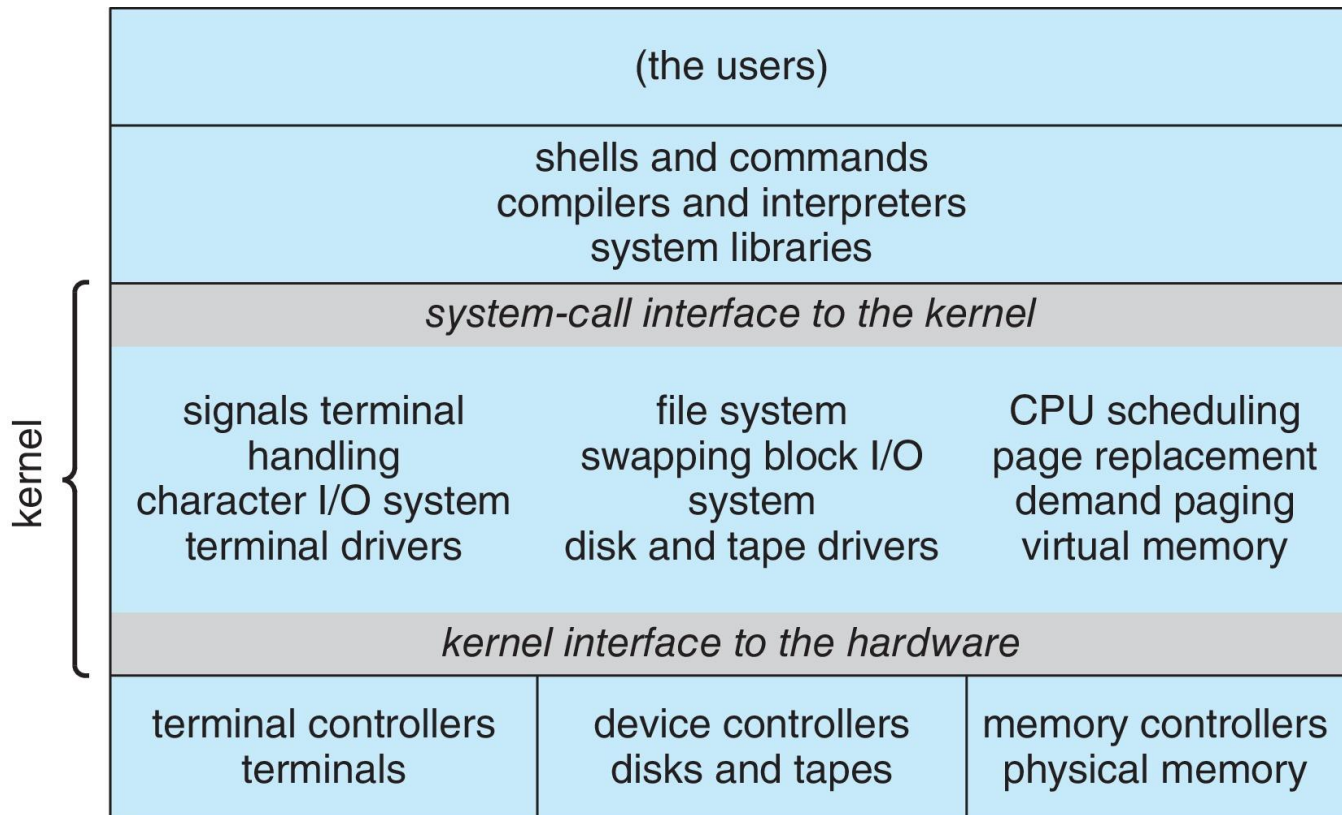
- UNIX – limitado por la funcionalidad del hardware, el sistema operativo UNIX original tenía una estructura limitada.
- El sistema operativo UNIX consta de dos partes separables
 - Programas de sistemas
 - El kernel
 - ▶ Consiste en todo lo que está debajo de la interfaz de llamada del sistema y por encima del hardware físico.
 - ▶ Proporciona el sistema de archivos, la programación del CPU, la gestión de la memoria y otras funciones del sistema operativo; una gran cantidad de funciones para un nivel





Estructura tradicional del sistema UNIX

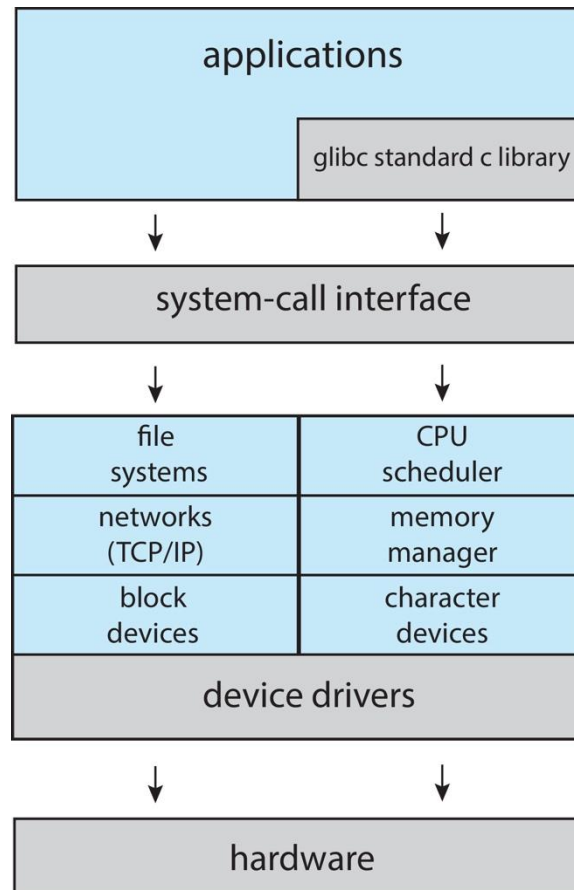
Más allá de lo simple pero no completamente estratificado





Estructura del sistema Linux

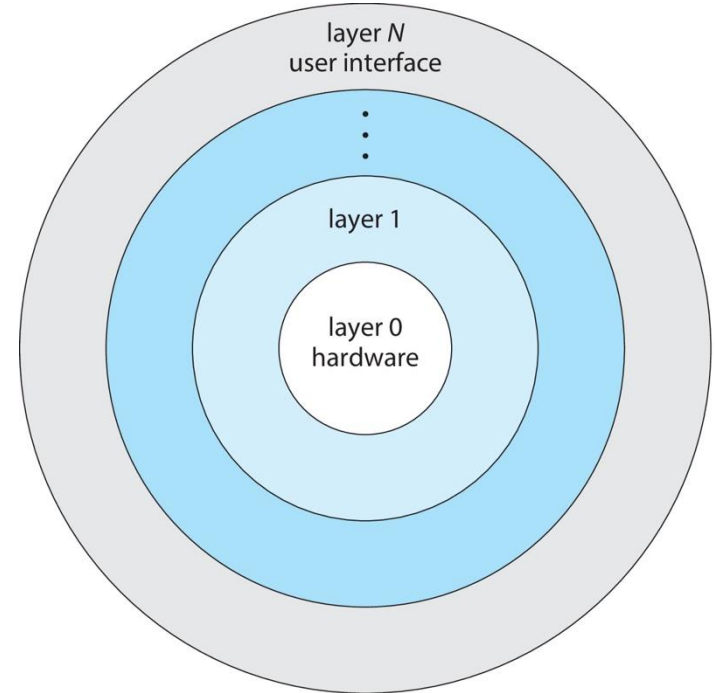
Diseño monolítico más modular





Enfoque en capas

- El sistema operativo se divide en varias capas (niveles), cada una construida sobre capas inferiores. La capa inferior (capa 0) es el hardware; la superior (capa N) es la interfaz de usuario.
- Con la modularidad, las capas se seleccionan de tal manera que cada una utiliza funciones (operaciones) y servicios de sólo capas de nivel inferior.





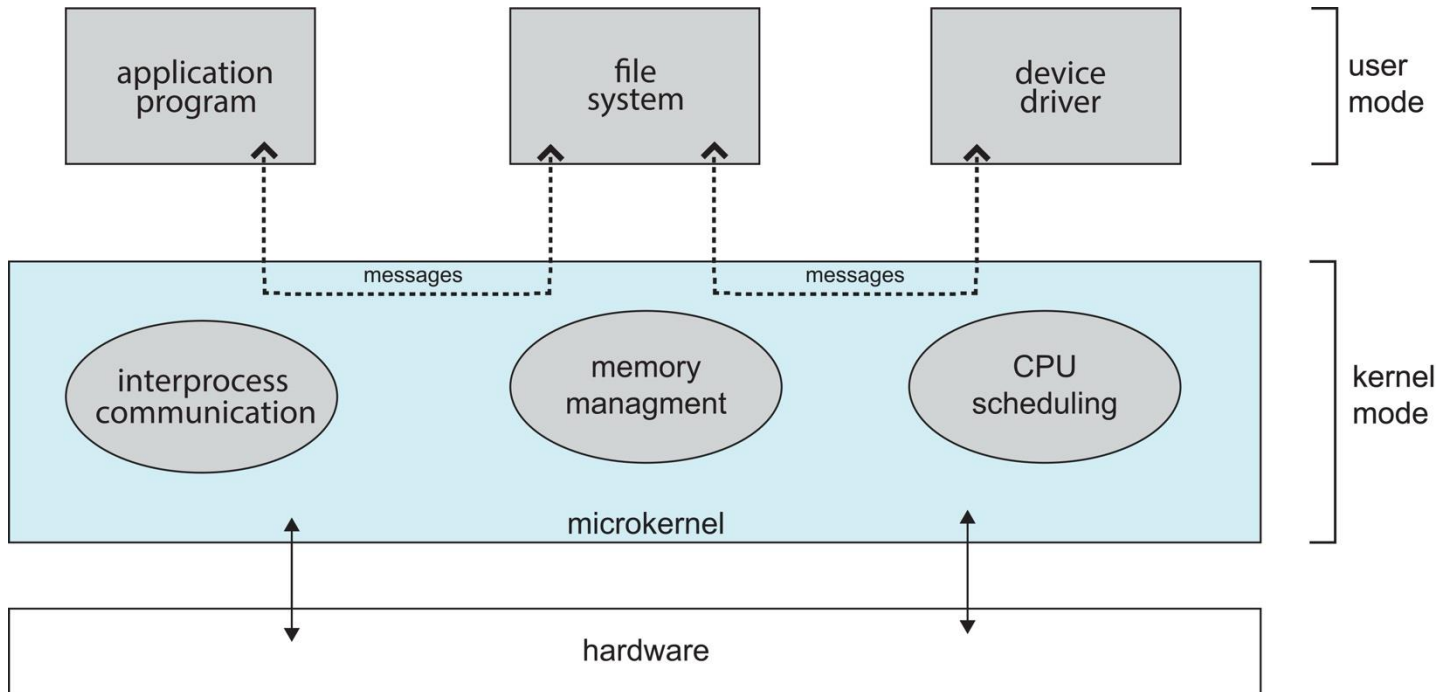
Microkernel

- Mueve tanto desde el kernel al espacio del usuario
- **Mach** es un ejemplo de **microkernel**
 - Kernel de Mac OS X (**Darwin**) basado parcialmente en Mach
- La comunicación se lleva a cabo entre los módulos de usuario mediante **paso de mensajes**
- Beneficios:
 - Es más fácil ampliar un microkernel
 - Es más fácil portear el sistema operativo a nuevas arquitecturas
 - Más confiable (se ejecuta menos código en modo kernel)
 - Más seguro
- Perjuicios:
 - Sobrecarga de rendimiento de la comunicación entre el espacio del usuario y el espacio del kernel





Estructura del sistema de microkernel





Módulos

- Muchos sistemas operativos modernos implementan **modulos cargables en kernel** (**LKM**)
 - Utiliza un enfoque orientado a objetos
 - Cada componente principal es independiente
 - Cada uno habla con los demás a través de interfaces conocidas.
 - Cada uno se puede cargar según sea necesario dentro del kernel.
- En general, similar a las capas pero con mayor flexibilidad.
 - Linux, Solaris, etc.





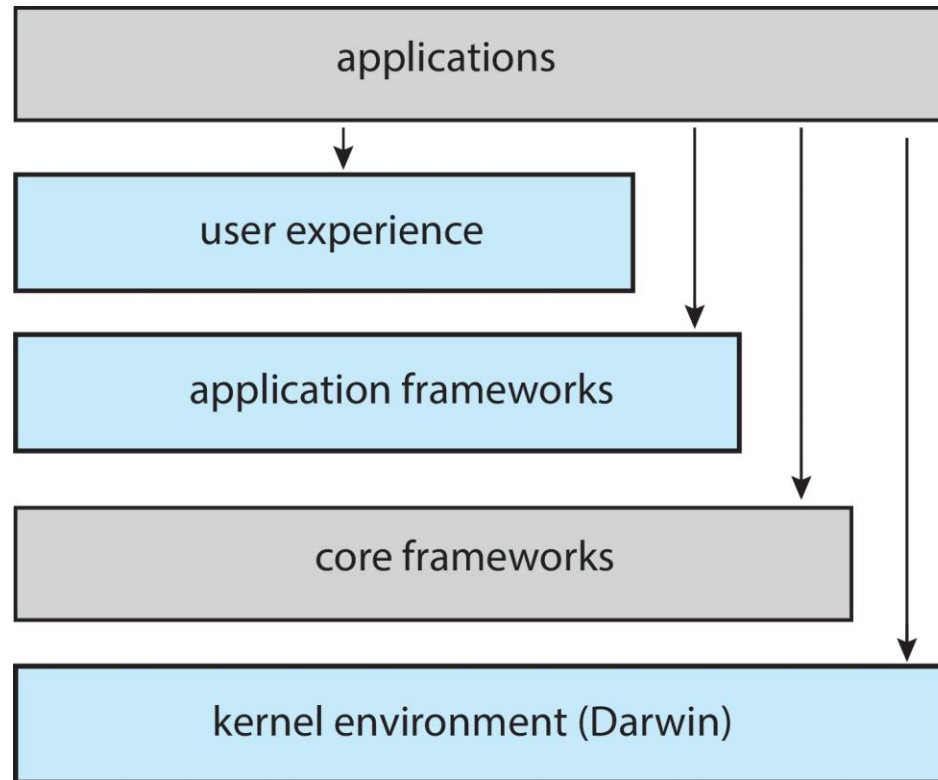
Sistemas híbridos

- La mayoría de los sistemas operativos modernos no son un modelo puro
 - El híbrido combina múltiples enfoques para abordar las necesidades de rendimiento, seguridad y usabilidad.
 - Kernels de Linux y Solaris en el espacio de direcciones del kernel, por lo que son monolíticos y modulares para la carga dinámica de funcionalidades.
 - Windows es mayoritariamente monolítico, además de microkernel para diferentes ***personalidades de subsistema***.
- Apple Mac OS X es híbrido, en capas, **Aqua** UI y ambiente de programación **Cocoa**
 - A continuación se muestra el núcleo que consta de microkernel Mach y partes BSD Unix, además de un kit de I/O y módulos cargables dinámicamente (llamados **extensiones de kernel**)



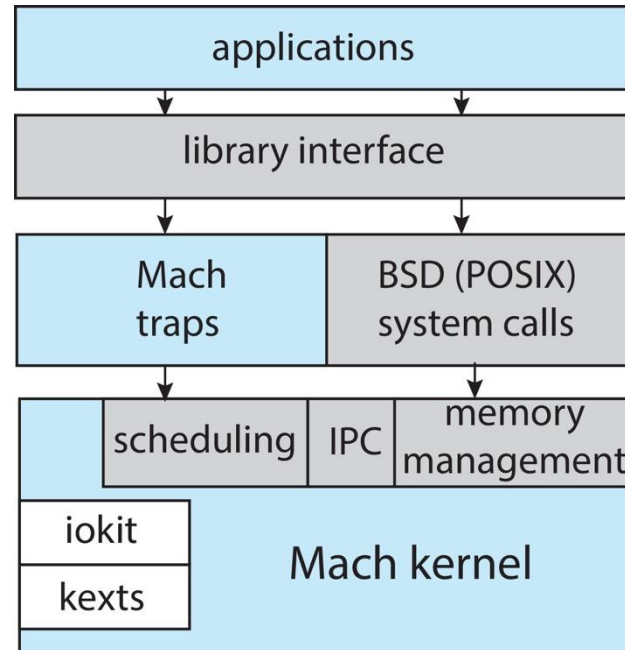


Estructura de macOS e iOS





Darwin





iOS

- Sistema operativo móvil de Apple para ***iPhone*** y ***iPad***
 - Estructurado en Mac OS X, funcionalidad añadida
 - No ejecuta aplicaciones OS X de forma nativa
 - ▶ También funciona en diferentes arquitecturas de CPU (ARM vs. Intel)
 - **Cocoa Touch** API de Objective-C para desarrollar aplicaciones
 - **Media Services** - capa para gráficos, audio, vídeo
 - **Core services** Proporciona computación en la nube, bases de datos
 - **Core OS**, basado en el kernel de Mac OS X

Cocoa Touch

Media Services

Core Services

Core OS





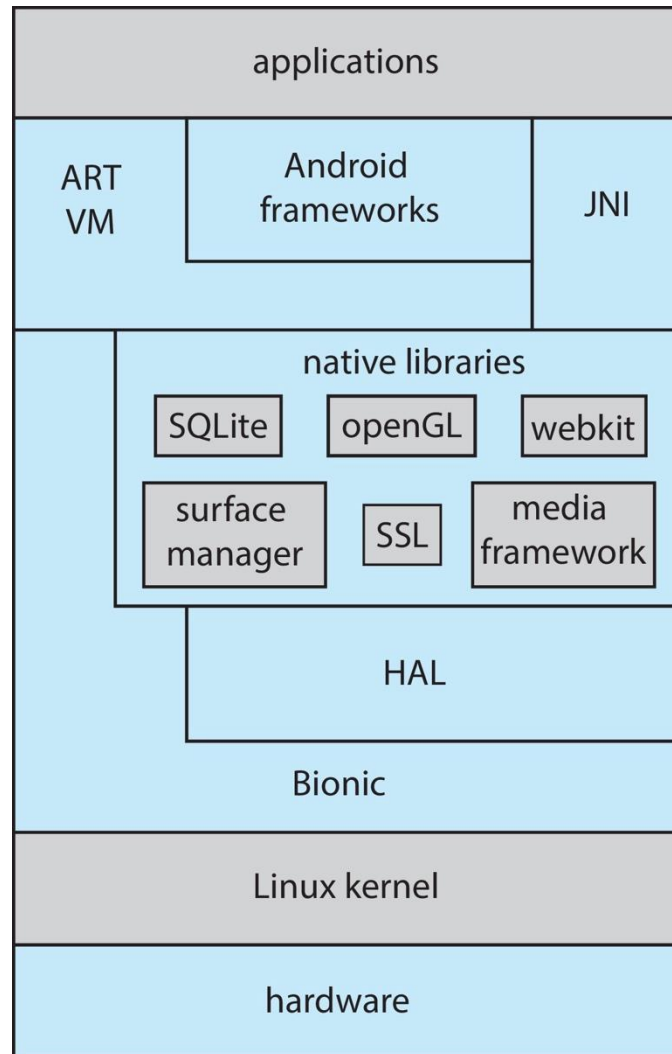
Android

- Desarrollado por Open Handset Alliance (principalmente Google)
 - Código abierto
- Pila similar a iOS
- Basado en el kernel de Linux pero modificado
 - Proporciona gestión de procesos, memoria y controladores de dispositivos.
 - Añade gestión de energía
- El entorno de ejecución incluye un conjunto básico de bibliotecas y una máquina virtual Dalvik
 - Aplicaciones desarrolladas en Java más API de Android
 - ▶ Archivos de clase Java compilados en código de bytes Java y luego traducidos a ejecutables que se ejecutan en Dalvik VM
- Las bibliotecas incluyen marcos de desarrollo para navegadores web (webkit), bases de datos (SQLite), multimedia y bibliotecas más pequeñas.





Arquitectura de Android





Construir y arrancar un sistema operativo

- Sistemas operativos generalmente diseñados para ejecutarse en una clase de sistemas con una variedad de periféricos.
- Comúnmente, el sistema operativo ya está instalado en la computadora comprada.
 - Pero puede construir e instalar otros sistemas operativos.
 - Si se genera un sistema operativo desde cero
 - ▶ Escribe el código fuente del sistema operativo
 - ▶ Configurar el sistema operativo para el sistema en el que se ejecutará
 - ▶ Compilar el sistema operativo
 - ▶ Instalar el sistema operativo
 - ▶ Arranque la computadora y su nuevo sistema operativo



Dudas?

